Планирование пути на графах

Современные методы управления роботами

Сергей Савин

2026

Содержание

- Планирование пути на графах
- Алгоритм Дейкстры: инициализация, исследование и обновление узлов, выбор следующего узла, восстановление пути
- Алгоритм A* и допустимые эвристики
- Итерация по значениям и уравнение Беллмана

Планирование пути на графах

Многие практические задачи планирования пути можно формализовать с помощью графов:

- Мы описываем среду как набор местоположений (узлов).
- Мы знаем, между какими местоположениями можно перемещаться (рёбра).
- Мы знаем стоимость перемещения из одного местоположения в другое (стоимость ребра).

Таким образом, планирование пути становится задачей поиска последовательности рёбер, соединяющих начальный узел с целевым узлом, такой, что сумма стоимостей рёбер минимальна.

Пути на графе

Для любого узла k пусть $\mathcal{N}(k)$ обозначает множество узлов, достижимых из k прохождением одного ребра. Они называются **смежными узлами**.

Путь можно представить как последовательность узлов, соединённых рёбрами. Каждый промежуточный узел имеет предшественника и преемника.

Оптимальный путь обладает важным свойством:

- Каждый промежуточный узел достигается из своего предшественника вдоль оптимального пути к этому узлу.
- Из каждого промежуточного узла оставшаяся часть пути к цели также оптимальна.

Алгоритм Дейкстры

Алгоритм Дейкстры находит кратчайшие пути от начального узла к другим узлам графа. Его также можно использовать для поиска кратчайшего пути к конкретному целевому узлу.

Ключевая идея — поддерживать наилучшую известную стоимость достижения каждого обнаруженного узла из начального узла.

Мы поддерживаем два множества:

- **Открытое множество:** узлы, которые были обнаружены, но ещё не полностью исследованы.
- **Закрытое множество:** узлы, стоимости кратчайших путей до которых уже установлены.

Мы предполагаем, что все стоимости рёбер неотрицательны.

Алгоритм Дейкстры: инициализация

Изначально обнаружен только начальный узел s:

$$\text{Open} = \{s\}, \qquad \text{Closed} = \emptyset.$$

Для каждого обнаруженного узла k мы поддерживаем предварительную стоимость $g(k)$: стоимость лучшего пути к k, найденного на данный момент. Начальная стоимость:

$$g(s) = 0.$$

Все остальные узлы изначально не обнаружены, их стоимости считаются бесконечными:

$$g(k) = +\infty, \qquad k \neq s.$$

Мы также сохраняем предшественника каждого обнаруженного узла для восстановления пути.

Алгоритм Дейкстры: исследование узла

Исследование узла k включает следующие шаги:

1. Проверить, является ли k целевым узлом.
2. Рассмотреть каждый смежный узел $m \in \mathcal{N}(k)$.
3. Переместить k из открытого множества в закрытое.

При рассмотрении смежного узла m мы проверяем, улучшает ли путь через k его наилучшую известную стоимость. Если m не был обнаружен, его предварительная стоимость инициализируется как

$$g(m) = g(k) + c(k, m),$$

где $c(k, m)$ — стоимость ребра из k в m.
Мы также записываем k как предшественника m и добавляем m в открытое множество.

Алгоритм Дейкстры: обновление узла

Иногда мы встречаем узел m, который уже был обнаружен. Пусть $g(m)$ — его текущая предварительная стоимость. Мы сравниваем её со стоимостью достижения m через узел k, который сейчас исследуется:

$$g_{\text{new}}(m) = g(k) + c(k, m).$$

Возможны два варианта:

- Если $g(m) \leq g_{\text{new}}(m)$, сохранить текущую стоимость и предшественника.
- Если $g(m) > g_{\text{new}}(m)$, обновить стоимость и назначить k новым предшественником m:

$$g(m) \leftarrow g_{\text{new}}(m).$$

Эта операция называется **релаксацией** ребра.

Алгоритм Дейкстры: выбор следующего узла

Ключевой шаг — выбор следующего узла для исследования. Мы выбираем узел k с наименьшей предварительной стоимостью среди всех узлов открытого множества:

$$k = \arg \min_{m \in \text{Open}} g(m).$$

Затем мы исследуем k и перемещаем его в закрытое множество.

- Если k — целевой узел, мы завершаем работу.
- В противном случае мы продолжаем выбирать и исследовать узлы.

Поскольку все стоимости рёбер неотрицательны, стоимость выбранного узла является стоимостью его кратчайшего пути.

Алгоритм Дейкстры: восстановление пути

После завершения работы алгоритма мы можем восстановить кратчайший путь к цели. Начиная с целевого узла, мы многократно находим его предшественника и добавляем его к пути.

Эквивалентно, для текущего узла m мы можем выбрать предшественника k, удовлетворяющего условию

$$k = \arg \min_{k:\, m \in \mathcal{N}(k)} \big(g(k) + c(k, m)\big).$$

Мы продолжаем, пока не достигнем начального узла s. Затем мы обращаем последовательность узлов, чтобы получить путь от s к цели.

Алгоритм Дейкстры: одна цель или все цели?

Алгоритм Дейкстры использует целевой узел только для определения момента завершения.

Если мы позволим алгоритму работать до тех пор, пока открытое множество не станет пустым, он вычислит стоимости кратчайших путей от начального узла до каждого достижимого узла графа.

- Затем мы можем выбрать любой достижимый узел в качестве цели и восстановить его кратчайший путь.
- Однако это может быть излишней работой, если нам нужен путь только к одной известной цели.

Мы можем использовать знание о цели, чтобы направлять поиск и исследовать меньше узлов.

Алгоритм A*

Алгоритм A* похож на алгоритм Дейкстры, но использует информацию о цели для направления выбора узлов для исследования.

В то время как Дейкстра выбирает узел с наименьшей предварительной стоимостью $g(k)$, A* выбирает узел с наименьшей оценочной полной стоимостью:

$$f(k) = g(k) + h(k).$$

Здесь:

- $g(k)$ — стоимость лучшего пути, найденного на данный момент от начала до узла k.
- $h(k)$ оценивает оставшуюся стоимость от узла k до цели.

Допустимая эвристика никогда не переоценивает истинную минимальную оставшуюся стоимость.

Итерация по значениям

Алгоритм Дейкстры начинается с начального узла и распространяет информацию о стоимости достижения других узлов.

В качестве альтернативы мы можем начать с цели и распространять информацию о стоимости достижения цели из каждого другого узла.

Эта величина называется **стоимостью до цели** (cost-to-go) и обозначается $V(k)$ для каждого узла k. Это ключевая идея **итерации по значениям**.

Итерация по значениям: инициализация

Мы инициализируем каждый узел k оценкой стоимости до цели $V(k)$.

- Для целевого узла g стоимость до цели равна нулю:

$$V(g) = 0.$$

- Для всех остальных узлов начальная оценка равна бесконечности:

$$V(k) = +\infty, \qquad k \neq g.$$

Затем мы итеративно обновляем эти оценки, используя стоимости перемещения к смежным узлам.

Итерация по значениям: обновления

Мы обновляем оценки стоимости до цели, используя **уравнение Беллмана**:

$$V(k) = \min_{m \in \mathcal{N}(k)} \big(c(k,m) + V(m)\big).$$

Это уравнение утверждает, что минимальная стоимость до цели получается выбором лучшего узла-преемника и добавлением стоимости перемещения к этому узлу.

На каждой итерации мы обновляем оценки для всех узлов, используя текущие оценки их соседей.

Мы повторяем этот процесс до тех пор, пока оценки не сойдутся.

Итерация по значениям: восстановление пути

После сходимости алгоритма мы можем восстановить кратчайший путь из любого начального узла s.

В каждом узле k мы выбираем преемника m, минимизирующего сумму стоимости ребра и стоимости до цели:

$$m = \arg \min_{m \in \mathcal{N}(k)} \big(c(k, m) + V(m) \big).$$

Мы переходим к выбранному преемнику и повторяем, пока не достигнем целевого узла g.

Литература

- Erik Demaine, Ronald Rivest, Srini Devadas – MIT: Introduction to Algorithms: Lecture Notes, lectures 15–18 (Shortest Paths and Dijkstra).
- Jason Ku, Justin Solomon – MIT: Introduction to Algorithms: Lecture 13, Dijkstra's Algorithm.
- MIT OpenCourseWare: Dynamic Programming and Stochastic Control: Lecture 10, Bellman's Equation and Value Iteration.

Слайды доступны на Github:

github.com/SergeiSa/Advanced-Control-2026

